

ACH2023 ALGORITMOS E ESTRUTURAS DE DADOS I

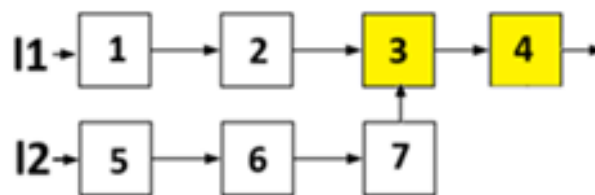
Semestre 2025-2 - Exercício prático 1 – Separação de listas compartilhadas

Responsável - Ivandré Paraboni

1. O objetivo do trabalho é implementar de forma correta e completa, em C ou C++, a função **separar** a seguir utilizando apenas listas ligadas simples como entrada, usando o modelo de código disponibilizado no sistema e-disciplinas.

```
void separar(LISTA* l1, LISTA *l2);
```

2. A função recebe como entrada dois ponteiros de listas ligadas l1 e l2 **não vazias** que compartilham **um ou mais** nós em seu segmento final. O exemplo a seguir ilustra uma situação em que duas listas possuem um segmento final composto de dois elementos compartilhados (em amarelo) começando no ponto de união 3.



3. As listas possuem chaves aleatórias do tipo inteiro, sem ordem e com possível repetição. Assim, o compartilhamento a ser considerado é referente ao mesmo segmento final de nós acessíveis tanto via l1 quanto l2, em amarelo, e não tem relação com o valor das chaves em si.
4. A função deve implementar duas operações para separar l1 de l2:
 - (i) Estender a lista l2 além do ponto de união, de modo a incluir os elementos inicialmente não acessíveis de l1 em ordem reversa, ou seja, terminando a lista l2 com o início de l1;
 - (ii) Ajustar a lista l1 de modo a conter apenas os elementos restantes não incluídos em l2.
5. No caso acima, as listas resultantes seriam l2={5,6,7,3,2,1} e l1={4}.
6. Observe que a resposta **sempre será composta de duas listas l1 e l2 sem elementos em comum**, e ambas terminando em NULL.
7. Se as listas compartilharem apenas um elemento (e.g., se não existir o elemento 4 no exemplo acima), l1 ficará vazia. Por outro lado, se as listas compartilharem todos os elementos (ou seja, se forem idênticas), l1 ficará com todos os elementos exceto o primeiro, e l2 ficará apenas com o primeiro elemento.
8. Restrições de implementação:
 - (a) *Não use nenhum vetor na sua implementação.* Estruturas auxiliares de implementação dinâmica podem ser utilizadas livremente, mas o uso de implementação estática invalida o trabalho.
 - (b) Não use variáveis globais. A função implementada deve definir localmente todas as variáveis e estruturas auxiliares, ou chamar funções auxiliares que o façam também em um escopo local.
 - (c) Não exiba nenhuma mensagem na tela, nem solicite que o usuário pressione nenhuma tecla etc.
 - (d) Não utilize código disponibilizado *online* ou em outras fontes, nem ferramentas de geração de código automático.
9. A função main() serve apenas para seus testes particulares, e não deve ser incluída na sua entrega.

10. O EP deve ser desenvolvido **individualmente ou em duplas previamente cadastradas no link abaixo até 31 de agosto** sem compartilhamento de código de espécie alguma. Não tente emprestar sua implementação para outros colegas, nem copiar deles, pois isso invalida o trabalho de **todos** os envolvidos. Seu trabalho é confidencial, e não pode ser compartilhado ou disponibilizado de forma alguma.
11. Seu programa será corrigido de forma *automática*, e por isso você não pode alterar as assinaturas da função solicitada, nem os tipos de dados ou especificações (*typedef*) do modelo fornecido.
12. O programa deve ser compilável no ambiente Windows 11 com DevC++ (mas sugere-se primeiro desenvolver em uma IDE mais funcional como Codeblocks). Se você não conseguiu fazer o programa compilar sem erros dentro do prazo, não entregue. Não é previsto nenhum tipo de avaliação além dos testes de execução, o que pressupõe um programa compilável e funcional.

O que/como entregar:

- A entrega será via upload no sistema **antes** da data estipulada.
- Entregue apenas um arquivo do tipo texto não compactado **NN.txt** onde NN são os dois dígitos do número do seu grupo conforme *link* acima (indivíduos também possuem um número), com o código da função principal e funções auxiliares que ela invoca, sem incluir main().
- Preencha as funções `nroUSP` e `Grupo()` do código exemplo disponível para que você seja identificado sem risco.

Prazos etc.:

O EP deve ser depositado no prazo definido na atividade cadastrada no sistema. Não serão aceitos EPs entregues depois do prazo, independentemente do motivo. Entregas no último dia são assim por conta e risco do responsável, e nenhum tipo de imprevisto de última hora (e.g., problemas de saúde, indisponibilidade de rede etc.) pode ser usado como justificativa para o atraso. O EP é uma atividade para ser desenvolvida ao longo de várias semanas, não nos últimos dias antes da entrega.

É responsabilidade do aluno que fez o *upload* do arquivo verificar se o mesmo foi corretamente recebido pelo sistema. Atrasos/falhas na submissão invalidam o trabalho realizado. Após o *upload*, verifique se você consegue abrir o arquivo depositado, e certifique-se de que é a versão correta do programa e que não está corrompido.

Critérios de avaliação:

A função será testada com uma série de chamadas repetidas e consecutivas, com diversas listas de como entrada. É assim importante assegurar que o seu programa funciona desta forma (por exemplo, chamando-o dentro de um laço *for*), e não apenas para um teste individual. Um teste é considerado correto se o resultado for exatamente como o esperado, ou incorreto em todo e qualquer caso divergente. Erros de alocação de memória ou compilação invalidam o teste, assim como a ausência de funções auxiliares necessárias para a execução do programa.

Este EP deve ser desenvolvido obrigatoriamente por *todos* os alunos de AED1. Sua nota é parte integrante da 1ª. avaliação e *não é* passível de substituição.